

Use Cases

Specifying behaviour from the outside in, one goal, one story at a time

By the end of this hour you will be able to:

- ✓ identify actors (human, machine, or clock) and the goals they pursue,
- ✓ draw a correct UML use-case diagram, including «include», «extend» and generalization,
- ✓ write a fully dressed use case: main success scenario, extensions and guarantees,
- ✓ judge whether a use case is written at the right level, in the right language.

The most expensive missing artifact in software

\$1.1 million
per day

estimated daily cost of the delay while Denver's finished-but-empty airport waited on its automated baggage system (opened 16 months late, Feb 1994; the system was never fully used and was later dismantled).

- The engineers were excellent. The budget was real. The technology existed.
- City-commissioned review, GAO briefing found: **requirements were gathered late and were changing**; airlines, the end users, were **not involved early enough**, and there was not enough time left to test.
- Standish Group CHAOS reports, edition after edition: **incomplete requirements** and **lack of user involvement** rank among the top reasons projects end 'challenged'.
- And remember last time: force customers to produce requirements too soon and they will **invent** them: wrong, incomplete, and expensive.

Great people, real money, working hardware. What was missing?

A shared, testable agreement on what the system must do, written as stories users can confirm. That agreement has a name: **the use case**.

#USE CASE



**BATCH
PROCESSING**



A use case = one goal, told as every story that can happen

- **UML 2.5:** a use case specifies a set of actions performed by the system that yields an **observable result of value** to an actor or other stakeholder.
- **Plain English:** one complete way the system is used to reach **one goal somebody cares about**, told from outside, as the actor experiences it.
- **Cockburn (2001):** 'a set of scenarios tied together by a common user goal', the happy path **plus every interesting way it can fail**.
- A use case is **text**. The diagram you will meet next is only its table of contents.

HISTORY — 30 SECONDS

- **mid-1980s:** Ivar Jacobson develops use cases at Ericsson (telephone switching systems).
- **1992:** published to the world in Jacobson et al., *Object-Oriented Software Engineering: A Use Case Driven Approach*.
- **1997:** use cases become part of the first UML standard, where they remain today.

Examples: name one goal you achieved with an app today. 'order coffee', 'submit assignment', 'unlock bike'.

Actors: roles at the boundary: not people, not jobs

- An **actor** is a **role** played by something **outside** the system that interacts with it.
- Three species:
 - **people in a role**: Customer, Operator, Student,
 - **external systems**: the Bank, a payment gateway, a map service (they want results too),
 - **the clock**: 'send overdue notices nightly' is triggered by time, so time is an actor.
- One human can play two actors; one actor is played by many humans. The role is what interacts.

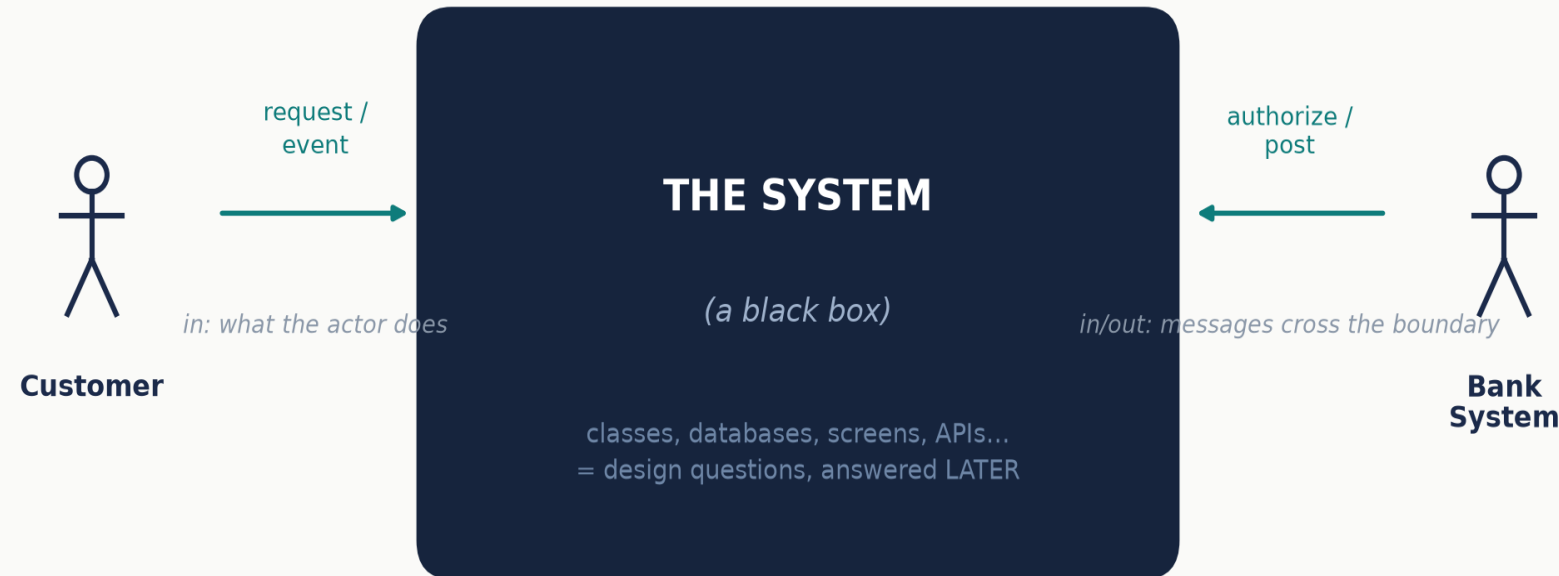
VOTE: TRUE OR FALSE (FINGERS)

- 1. An actor is always a human.
- 2. One person can play two actors.
- 3. A use case can have several actors.
- 4. Your app's database is an actor.

Boundary test: if you cannot draw it outside the system rectangle, it is not an actor, it is a module. The database lives inside; the bank's authorization service lives outside.

The black-box rule: describe the conversation, not the machinery

A use case describes **ONLY** what crosses the boundary — never what is inside.



X leaked step

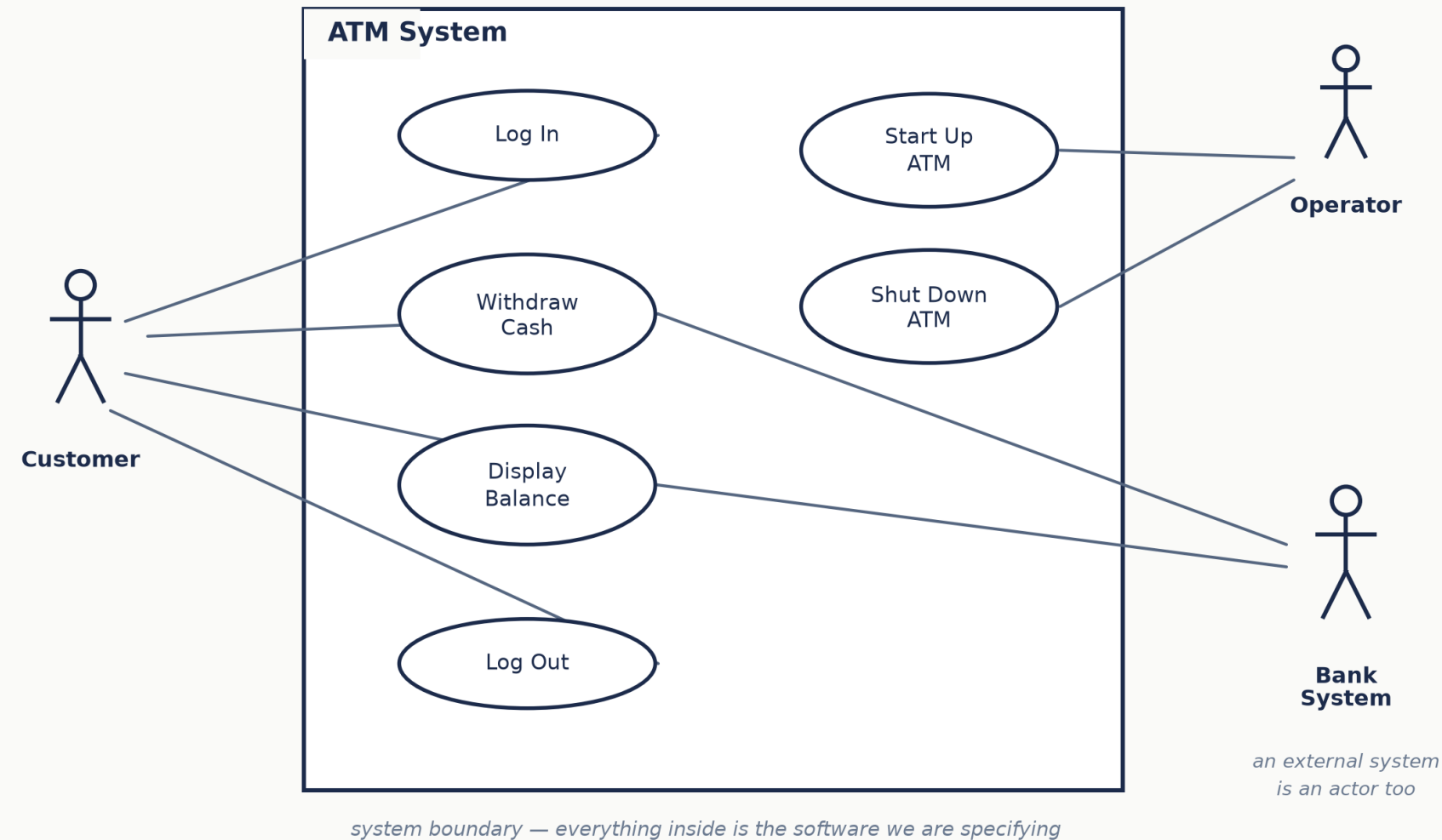
"The system runs `SELECT balance FROM accounts WHERE id=42` and renders `balance.jsp`."

✓ black-box step

"The system displays the customer's current balance."

If a step mentions a screen, a table, a class or an API, you opened the box. Close it. How it works is a design question, and design is a later lecture.

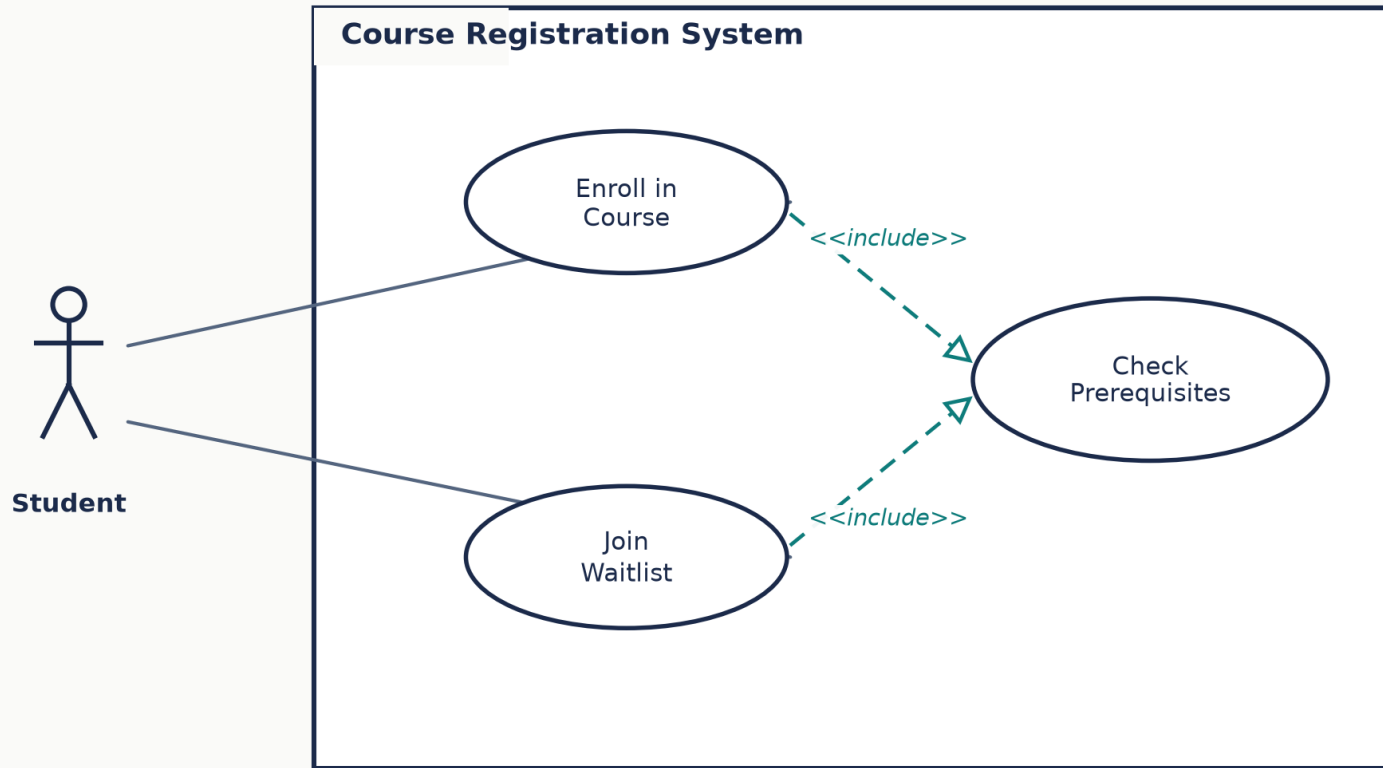
The use-case diagram: a map, not the territory



READING THE MAP

- **stick figure** = actor (a role outside)
- **ellipse** = one use case (one goal)
- **rectangle** = system boundary
- **plain line** = actor participates
- Names: **verb + object**, at the user's goal level:
 - ✓ **Withdraw Cash**
 - ✗ **ATM Stuff** (vague)
 - ✗ **Click Withdraw Button** (GUI detail)
- No sequence, no arrows between actors: order lives in the text, not here.

«include»: behaviour that ALWAYS happens, written once

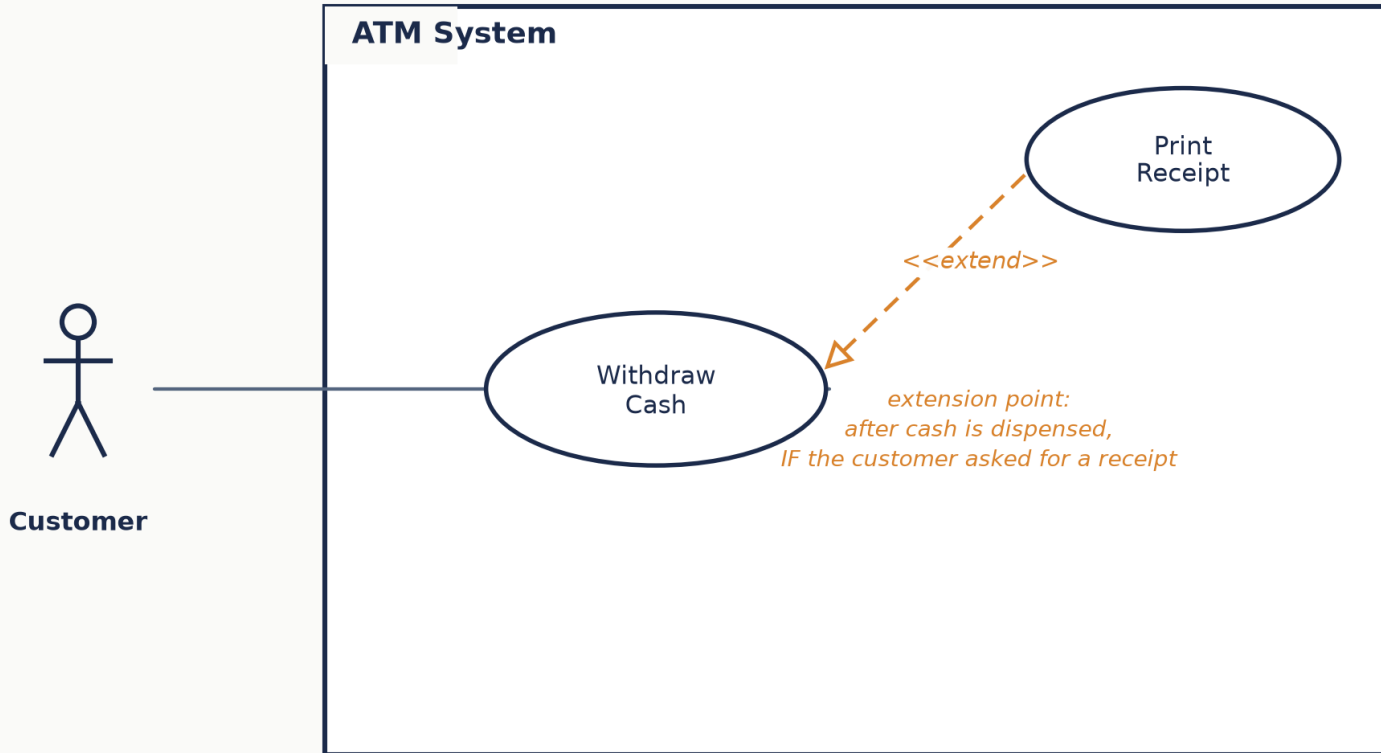


«include» = the base use case ALWAYS executes the included behaviour (here: both paths must check prerequisites)

- **Meaning:** the base use case always executes the included behaviour as part of itself.
- **Arrow:** base → included (dashed, open head).
- **Use it when:** several use cases repeat the same steps, or a step-set is meaningful on its own.
- **Not** a function call: the included use case still reads as user-visible behaviour.
- Larman's advice: use it **sparingly**, most good models have few.

Here both Enroll in Course and Join Waitlist must check prerequisites, so the check is written once and included twice.

«extend»: behaviour that happens SOMETIMES, on a condition



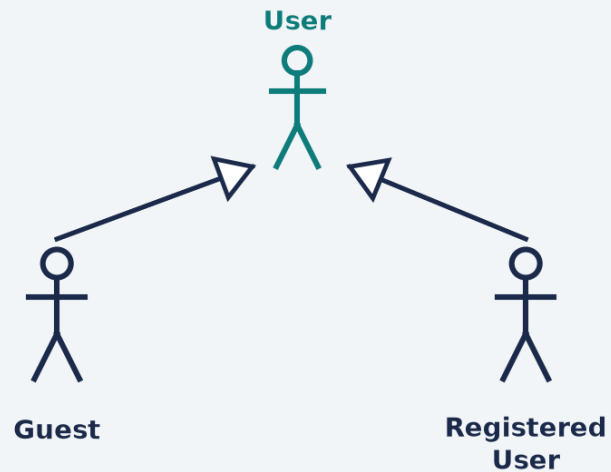
«extend» = OPTIONAL behaviour, runs only under a condition — arrow points FROM the extension TO the base

- **Meaning:** optional / conditional behaviour inserted at an **extension point**, only if its condition holds.
- **Arrow:** extension → base. Yes, the 'surprising' direction, the optional piece points at the story it joins.
- **Use it for:** optional features and rare exception paths you want out of the main story.
- The base use case must stay **complete and meaningful** without the extension.

Withdrawing cash works perfectly without a receipt, the receipt is an extra, so it extends.

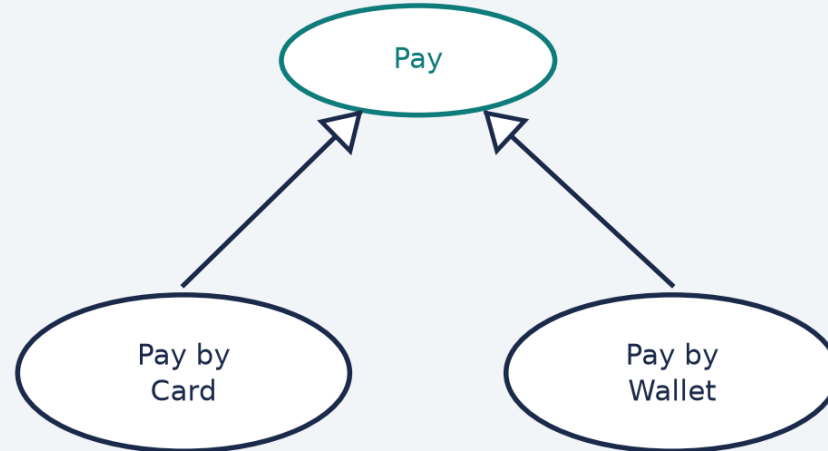
Generalization: the 'is a kind of' relation, again

Actor generalization



*child actors can be used
anywhere the parent is*

Use-case generalization



*child inherits the parent's behaviour
and may add / override steps*

- **Actors:** a child role can stand anywhere the parent role appears.
- **Use cases:** a child inherits the parent's steps and may add or override some.
- Same hollow-triangle notation you will use for classes, UML is one language.
- If you cannot say 'is a kind of', you do not have generalization.

Three relations, ten-second recall

relation	runs when...	arrow points...	our example
«include»	always, as part of the base	base → included	Enroll in Course → Check Prerequisites
«extend»	only if condition holds at extension point	extension → base	Print Receipt → Withdraw Cash
generalization	child replaces parent, inheriting its steps	child → parent	Pay by Card → Pay

- 'Apply a promo code' joins 'Check Out' only when the customer has one → ?
- 'Verify PIN' is repeated by 'Log In' and 'Change PIN' → ?
- 'Save the row to the accounts table' → ?

Mini-exercise: Campus Shuttle Tracker

The system: riders watch live shuttle positions and get 'your shuttle is 4 min away' alerts; the driver's phone feeds a position every 20 seconds; map tiles come from an external map service; every 5 minutes the system recomputes ETAs and pushes changed alerts.

- List the **actors**, at least one must be non-human. Which one is the clock?
- Name **three use cases** at sea level, verb–object (no 'Manage Stuff').
- Add **one «include» or «extend»** with the arrow pointing the right way.

Scribble it. Ugly is fine, wrong arrows are gold, we learn from them out loud.

Three formats: the same use case at three dials (Cockburn)

BRIEF

one paragraph

when: early exploration, backlog chats

"Customer logs in, picks an amount, the ATM checks with the bank and dispenses \$20-multiples up to \$500, showing the new balance; on any decline the customer gets a message and no cash moves."

CASUAL

a few informal paragraphs

when: team already shares the context

Same story, a couple of paragraphs, no numbering, failures mentioned in passing, enough to remind, not enough to contract.

FULLY DRESSED

all sections, numbered

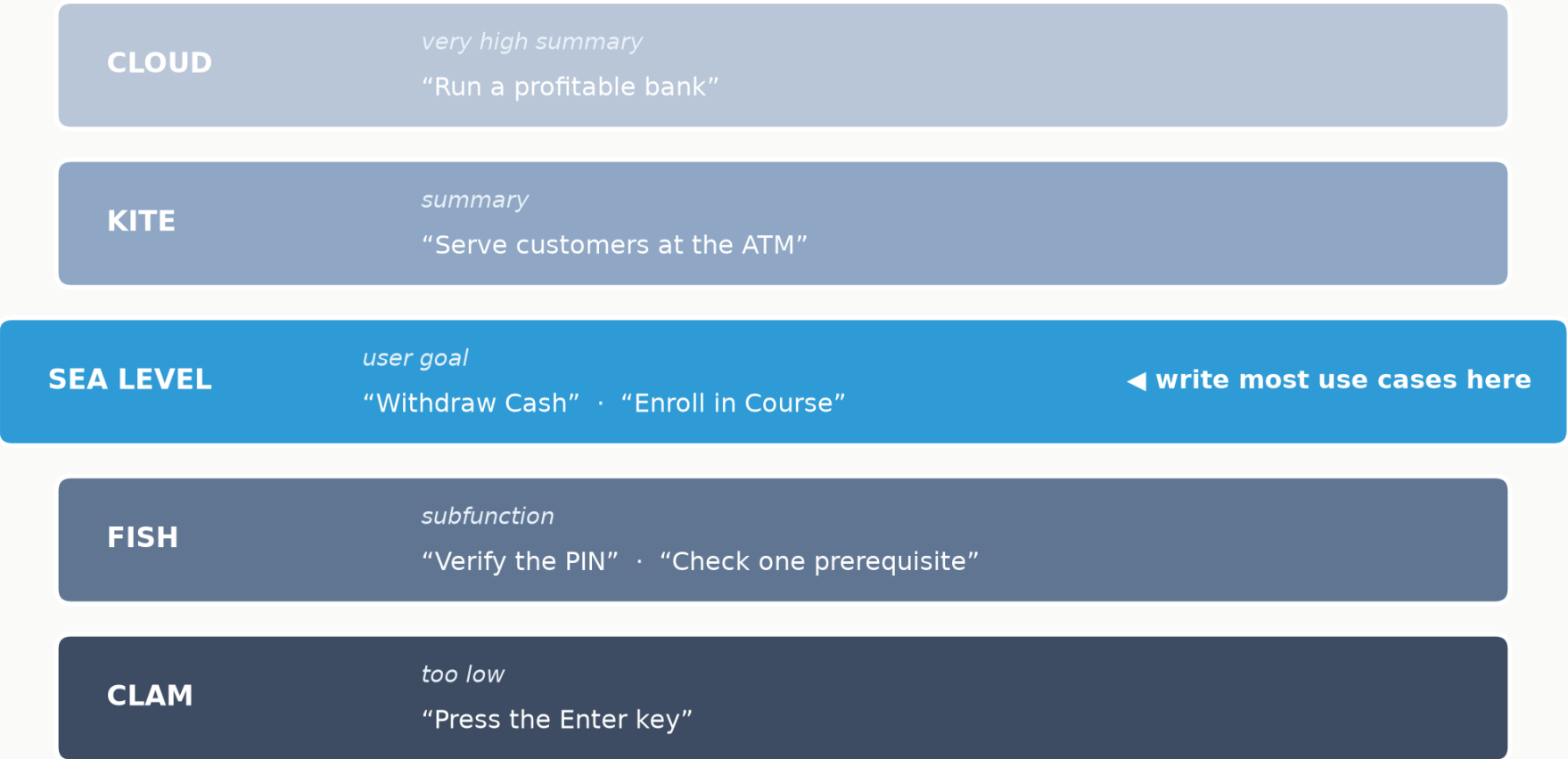
when: the behaviour is a contract, projects, exams, your sprint today

Goal, actors, preconditions, trigger, main success scenario, extensions, guarantees, special requirements. The next four slides build one line by line.

A use case may start brief and grow fully dressed as stakes rise, the format is a dial, not a cage.

At what altitude are you writing?

Goal levels (A. Cockburn, Writing Effective Use Cases, 2001)



CLASSIFY!

- "Run a profitable bank"
- "Withdraw Cash"
- "Validate the card checksum"
- "Tap the green button"
- "Shut Down ATM"

Test for sea level: one person, one sitting, leaves happy — the goal the user would state.

The last one is the trap: sea level, the OPERATOR's user goal, not the customers. Sea level is per actor, not per system.

The fully dressed template, ten clauses of a contract

1 Name & ID verb-object, e.g. UC-01 Withdraw Cash	2 Goal & summary who gets what, in one or two lines
3 Actors, scope, level primary + supporting; which system; sea level	4 Preconditions what must ALREADY be true (may cite other use cases)
5 Trigger the event that starts the story (actor action or time)	6 Main success scenario 3–9 numbered actor–system exchanges, no branching
7 Extensions every interesting failure, numbered 2a, 3b... with resume/exit	8 Postconditions success guarantee AND minimal guarantee on failure
9 Special requirements the NFRs specific to THIS use case	10 Glossary & notes domain words, frequencies, open questions

This is our course template: Cockburn's fully dressed format in uniform. Clauses 6–7 are the heart; the rest is the contract around it.

UC-01 Withdraw Cash ①, header, preconditions, trigger

Name: UC-01 Withdraw Cash (verb-object ✓)

Goal: A logged-in customer obtains cash from their account, in €20 bills, up to \$500 per day.

Summary: Customer chooses an amount; the ATM settles it with the bank; cash and a fresh balance come out.

Primary actor: Customer (initiates, receives the value)

Supporting actor: Bank System (authorizes, debits, wants a correct, logged transaction)

Scope & level: ATM System · user goal (sea level)

Preconditions: ATM is started up (see UC-05 Start Up ATM). Customer has inserted a valid card and entered the correct PIN (see UC-06 Log In).

Trigger: The customer selects the “Withdraw Cash” option.

- Preconditions are **already true** when the story starts. If the use case must check it, it is a step or an extension, not a precondition.
- Preconditions may **cite other use cases** (UC-05, UC-06): traceability for free.
- Trigger = the **first domino**: an actor action or a clock tick. Not 'the system starts'.
- Goal states **value**: cash in hand, measurable, testable, signable.

UC-01 Withdraw Cash ②, the main success scenario

1. The system prompts the customer for a withdrawal amount.
2. The customer chooses a listed amount or enters one and confirms it.
3. The system requests authorization from the bank; the bank confirms sufficient funds.
4. The system dispenses the amount in €20 bills.
5. The bank debits the account; the system displays the new balance (see UC-02 Display Balance).

- Each step = **one observable exchange**. 'The system processes' is not a step, it is a shrug.
- **User language only**. No screens, tables, classes, APIs.
- **3–9 steps** ideal, ~10 max. More means you have two use cases.
- **No branching here**. Every 'unless...' moves to an extension (next slide).
- Cross-references (UC-02) keep each story short, same instinct as «include».

UC-01 Withdraw Cash ③, extensions: where stories survive failure

2a. Amount is not a multiple of €20.

2a1 The system displays “amounts are dispensed in €20 bills”.

2a2 Resume at step 1.

2b. Amount exceeds the €500 daily limit.

2b1 The system displays the limit. 2b2 Resume at step 1.

3a. The bank declines (insufficient funds).

3a1 The system displays the decline and offers the balance (UC-02). 3a2 EXIT: no cash, account unchanged.

4a. The dispenser jams or is empty.

4a1 The system displays an error and logs a fault for the operator. 4a2 EXIT: account NOT debited.

- ID = **step number + letter**: the extension hangs on the step it interrupts.
- Every extension ends in exactly one of: **resume at n / retry n / exit with an outcome**.
- The exit lines ARE the minimal guarantee: money never silently vanishes.
- An unhandled 'unless' is a **bug you approved in writing**.

UC-01 Withdraw Cash ④, guarantees, special requirements, glossary

Success guarantee (postconditions)

Customer holds the requested cash · account debited by exactly that amount · new balance displayed to the customer.

Minimal guarantee (on any failure)

No cash dispensed AND no debit, OR both · customer told the reason · fault logged for the operator.

Special requirements (NFRs of THIS use case)

Respond to each customer input within 15 s · messages in the customer's chosen language · card number and PIN never displayed or printed.

Glossary

customer = holder of a valid card and account · bank (system) = external system maintaining accounts and balances · €20 bill = smallest dispensable unit.

- **Success guarantee** = postconditions of the happy path.
- **Minimal guarantee** = the promise kept on EVERY exit, including 3a/4a. This is what customers actually trust.
- NFRs listed HERE are the ones specific to this use case; global NFRs live in the spec's NFR section (last lecture).
- Glossary kills the cheapest bugs: two people, two meanings, one word.

Same skeleton, new domain, UC-07 Enroll in Course (60-second read)

Goal: Student holds a seat in an open, eligible course offering.

Actors: Student (primary) · Billing System (supporting, external)

Preconditions: Student logged in · enrollment window open · no administrative hold.

Trigger: Student requests enrollment in a chosen offering.

Main success: 1 System shows the offering. 2 Student confirms. 3 System verifies prerequisites. 4 System verifies a free seat. 5 System checks timetable clashes. 6 System assigns the seat and updates the fee record. 7 System shows the updated schedule.

Extensions: 3a prerequisites missing → show them, EXIT. 4a offering full → offer UC-08 Join Waitlist; if accepted, waitlist and EXIT. 5a clash → show clashing course, EXIT. 6a Billing unreachable → seat kept, fee queued for retry, student informed.

Guarantees: Success: seat assigned, enrollment + fee recorded. Minimal: no seat consumed, student told the exact reason.

Special req.: Enrollment completes within 5 s for 95% of requests at peak (500 concurrent students).

Hunt while reading: which actor is a machine? which extension hands over to another use case? which extension is a system being unavailable?

Six rules that separate use cases from noise

1

Name = verb + object at goal level

Withdraw Cash ✓ · ATM Stuff ✗ · Click Withdraw ✗

2

Write in the customer's language

the customer signs this document, not your codebase

3

Every step moves the ball

observable progress; 'the system processes' is banned

4

One goal per use case, at sea level

if two actors leave happy for different reasons, split

5

Main path ≤ ~10 steps; exceptions → extensions

long main paths are two use cases holding hands

6

Black box: no GUI, classes, tables, APIs

opened box = design leaking into the contract

Cockburn's frame: a use case is a contract for behaviour between stakeholders. Write it like one, specific enough to test, plain enough to sign.

Flaw hunt

A

Use case name: “ATM Interface”

B

Step: “The customer clicks the green Withdraw button on screen 3.”

C

Step: “The system validates the amount with regex `^[0-9]+$` and calls `AccountService.debit()`.”

D

Use case name: “Handle Transactions”

E

Main path says “the bank approves” — and nowhere in the document does the bank ever decline.

F

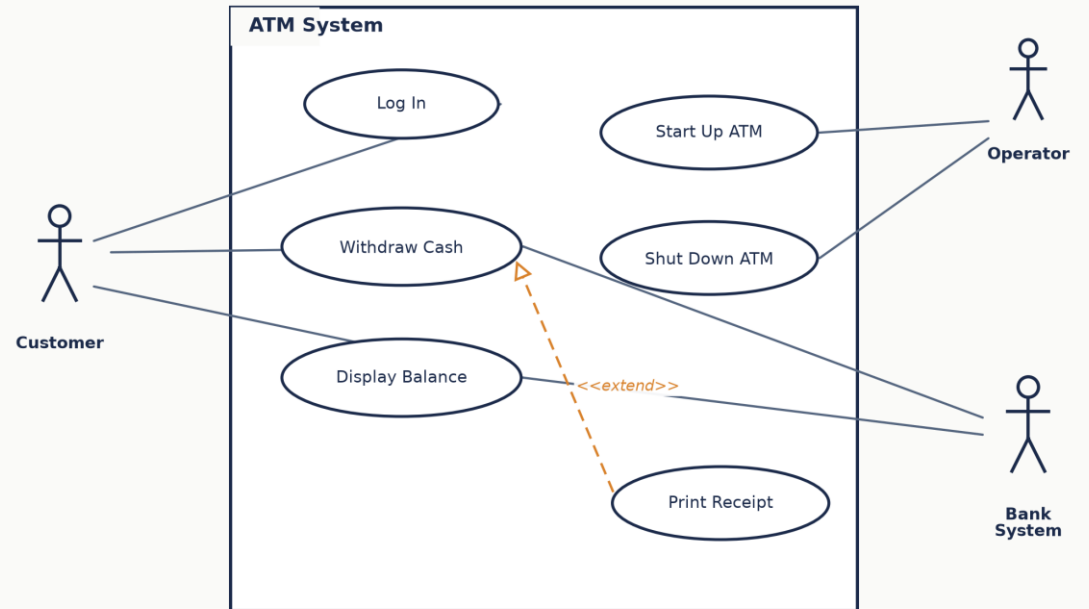
Step: “The system processes the request.”

Diagrams as code: model in text, render anywhere

```
@startuml
    left to right direction
    actor Customer
    actor Operator
    actor "Bank System" as Bank

    rectangle "ATM System" {
        usecase "Log In" as UC1
        usecase "Withdraw Cash" as UC2
        usecase "Display Balance" as UC3
        usecase "Print Receipt" as UC4
        usecase "Start Up ATM" as UC5
        usecase "Shut Down ATM" as UC6
    }

    Customer --> UC1
    Customer --> UC2
    Customer --> UC3
    Operator --> UC5
    Operator --> UC6
    Bank --> UC2
    Bank --> UC3
    UC4 ..> UC2 : <<extend>>
@enduml
```



Same model, two forms. Text lives in git: diffable, reviewable in a pull request, re-renderable by CI (plantuml.com).

Later in this course: the use case becomes an object

```
# withdraw_cash.py - the code twin of use case UC-01
from dataclasses import dataclass
from decimal import Decimal

@dataclass(frozen=True)
class WithdrawCashRequest:          # data carried by the trigger
    card_id: str
    amount: Decimal

class WithdrawCashUseCase:
    """One fully dressed use case = one application service."""
    def __init__(self, session, bank, dispenser):  # ports, not details
        self.session, self.bank, self.dispenser = session, bank, dispenser

    def execute(self, req: WithdrawCashRequest) -> "WithdrawCashResponse":
        if not self.session.logged_in():           # PRECONDITION
            return WithdrawCashResponse(ok=False, reason="Not logged in")
        if req.amount % 20 != 0:                   # EXTENSION 2a
            return WithdrawCashResponse(ok=False, reason="Multiples of $20 only")
        if req.amount > Decimal(500):              # EXTENSION 2b
            return WithdrawCashResponse(ok=False, reason="Daily limit is $500")
        balance = self.bank.authorize(req.card_id, req.amount)  # step 3
        if balance is None:                        # EXTENSION 3a
            return WithdrawCashResponse(ok=False, reason="Bank declined")
        self.dispenser.dispense(req.amount)        # step 4
        return WithdrawCashResponse(ok=True, dispensed=req.amount,
                                    new_balance=balance - req.amount)
```

- **1 use case → 1 application service** (a.k.a. interactor): Clean Architecture, R. Martin 2017.
- Dependencies enter as **ports** (session, bank, dispenser), hexagonal architecture, Cockburn 2005.
- precondition → **guard clause**
- extension → **early return with a reason**
- main-path step → **port call**
- The use case text stays the **spec**; this class is one valid reading of it.

Extensions are a test plan, written before code

```
# test_withdraw_cash.py - every extension is already a test
from decimal import Decimal
from withdraw_cash import WithdrawCashRequest

def test_main_success_scenario(uc):
    r = uc.execute(WithdrawCashRequest("card-1", Decimal("100")))
    assert r.ok and r.dispensed == Decimal("100")

def test_extension_2a_not_multiple_of_20(uc):
    r = uc.execute(WithdrawCashRequest("card-1", Decimal("50")))
    assert not r.ok and "Multiples of $20" in r.reason

def test_extension_2b_over_daily_limit(uc):
    r = uc.execute(WithdrawCashRequest("card-1", Decimal("600")))
    assert not r.ok and "500" in r.reason

def test_extension_3a_bank_declines(uc):
    r = uc.execute(WithdrawCashRequest("card-1", Decimal("500")))
    assert not r.ok and "declined" in r.reason

def test_precondition_not_logged_in(uc_logged_out):
    r = uc_logged_out.execute(WithdrawCashRequest("card-1", Decimal("100")))
    assert not r.ok and "Not logged in" in r.reason
```

- requirement → **use case** → test → code → passing build.
- Last lecture you met **verifiable** and **traceable**: this chain is what they look like in practice.
- Count them: 5 clauses on the use case, 5 tests. An extension with no test is a promise with no witness.

pytest naming shown is conventional (test_<behaviour>); fixtures 'uc' omitted for space.

The lecture summary

- A use case = **one actor goal**, told as scenarios: main success + every interesting failure, **black box**.
- Actors are **roles outside the boundary**, people, external systems, or the clock.
- The diagram is the **map**; the text is the **territory**. Verb–object names, sea level.
- **include** = always · **extend** = sometimes, at an extension point · **generalization** = is-a.
- Fully dressed = contract: preconditions, trigger, main success scenario, extensions, **success + minimal guarantees**.
- Use cases seed **design objects and tests**, and sit in your Functional Specification beside the FRs/NFRs.

Team sprint

Deliverables (one sheet / one doc):

- ▶ **Diagram:** ≥ 4 use cases, ≥ 2 actors with ≥ 1 non-human, ≥ 1 «include» or «extend» with correct arrow direction, boundary + system name.
- ▶ **One fully dressed use case** using the ten-clause template (next slide).
- ▶ **Self-check:** every name verb–object & sea level? zero GUI/tech words? every 'system approves' has its 'system declines'?

Product cards:

- A Campus Café app, order ahead, pay, collect
- B Campus Shuttle tracker (the one from slide 12!)
- C Gym class booking & QR check-in
- D Library self-checkout kiosk
- E Food delivery, order, rider, live tracking
- F Exam slot booking & fee payment

Hint actors to consider: payment gateway · map service · printer/kiosk hardware · nightly scheduler.

Worksheet skeleton

UC-__ _____ (verb + object)

Goal: _____

Primary actor: _____ Supporting actors: _____

Scope & level: _____ system · user goal

Preconditions: 1) _____ 2) _____ (may cite UC-__)

Trigger: _____

Main success scenario: 1 _____ 2 _____ 3 _____ 4 _____ 5 _____

Extensions: 2a ... 3a ... 4a ... (each: resume n / retry n / EXIT + outcome)

Success guarantee: _____

Minimal guarantee: _____

Special requirements: _____

Glossary: _____

Same ten clauses as slide 15, in writing order. If a clause feels empty, ask: 'and if not?', something will appear.

References & further reading

- I. Jacobson, M. Christerson, P. Jonsson, G. Övergaard: Object-Oriented Software Engineering: A Use Case Driven Approach. Addison-Wesley, 1992. (birth of use cases in print)
- A. Cockburn: Writing Effective Use Cases. Addison-Wesley, 2001. (formats, goal levels, guarantees; the quoted definition)
- C. Larman: Applying UML and Patterns, 3rd ed. Prentice Hall, 2004, ch. 6. (use cases are text; include/extend used sparingly)
- OMG: Unified Modeling Language (UML), v2.5.1, 2015. (definitions of use case, actor, include, extend, generalization)
- R. C. Martin: Clean Architecture. Prentice Hall, 2017. (use cases as interactors / application services)
- A. Cockburn: Hexagonal Architecture, 2005 (alistair.cockburn.us). (ports & adapters shown in the code twin)
- U.S. GAO, RCED-95-35BR (1994) & the Callear review for the City of Denver; TUM case-study notes. (Denver airport baggage-system facts)
- Standish Group: CHAOS reports (multiple editions). (requirements among top challenge factors)
- PlantUML: plantuml.com. (diagram-as-code demo)